Energy Demand Forecasting + RL Loading Scheming

Euan Roie Nicolas
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 956 832 1369
yuro.roie@gmail.com

Jan Nolasco
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 962 829 0338
jannolasco05@gmail.com

Jared Gabriel Vitero
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 956 507 7873
jaredvitero@gmail.com

Jeremiah Carbungco
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 969 646 2760
jeremiah.carbungco29@gmail.c
om

1. PROBLEM AND MOTIVATION

Energy providers and grid operators face a persistent and compounding challenge: electricity demand is inherently dynamic, shaped by behavioral patterns, seasonal cycles, and unpredictable events, yet infrastructure planning and load dispatching decisions must often be made hours or days in advance. Conventional rule-based scheduling systems respond to historical averages and fixed heuristics, leaving significant room for inefficiency as real conditions deviate from baseline assumptions. When supply and demand fall out of alignment, the consequences range from increased operational costs to unnecessary reliance on expensive peaker plants and elevated carbon emissions.

This project tackles two main issues. First, there is a forecasting challenge: household-level short-term electricity demand is a noisy, non-stationary time series that encodes intricate temporal dependencies, including weekly rhythms, seasonal drift, daily consumption cycles, and appliance-specific signatures. On this type of problem, hybrid deep learning models that combine recurrent and convolutional architectures have shown excellent performance, and ensemble approaches that combine LSTM with statistical models like Prophet have produced R2 values as high as 0.98 on hourly electricity data .Second, there is a scheduling problem: how should a system automatically determine when to postpone deferrable loads, like washing machines, dishwashers, or EV chargers, to off-peak hours in order to lower proxy energy costs without going against user comfort constraints, given a trustworthy demand forecast? Reinforcement learning is motivated by framing this as a sequential decision-making problem under uncertainty.

This project proposes an integrated academic prototype that addresses both challenges in sequence. A time-series forecasting module, implemented as both an LSTM encoder-decoder and a temporal convolutional network (TCN), generates short-term demand predictions from historical consumption data. These predictions are passed as input to a reinforcement learning (RL) load scheduling agent that learns, through repeated simulation, a policy for deferring loads to cost-effective windows. A natural language processing (NLP) component bridges the system and its users by translating raw forecast values and agent decisions into plain-language summaries with explicit override options, ensuring that automation never silently overrides user preferences.

The project uses the UCI Individual Household Electric Power Consumption dataset as its empirical foundation and operates entirely as an offline simulation. No real devices are controlled, and no real users are affected. The goal is to demonstrate that a fully integrated forecasting-scheduling-explanation pipeline can be implemented, evaluated, and held to rigorous ethical standards within an academic research context.

2. DATASET

2.1 Source and Description

The dataset used in this project is the Individual Household Electric Power Consumption Dataset, contributed by Georges Hébrail and Alice Bérard and hosted by the UCI Machine Learning Repository (DOI: 10.24432/C58K54). The dataset contains time-series measurements of electricity consumption recorded from a single household located in Sceaux, France, over a period spanning December 2006 to November 2010, at a one-minute sampling interval. The original dataset comprises approximately 2,075,259 one-minute records across seven numeric variables: Global_active_power, Global_reactive_power, Voltage, Global_intensity, and three sub-metering channels (Sub_metering_1, Sub_metering_2, Sub_metering_3).

For the purposes of this project, the raw one-minute records were resampled to hourly resolution by computing the mean of all non-null readings within each hour. After cleaning and resampling, the working dataset contains 34,168 hourly records with zero missing values. The forecasting target is Global_active_power, measured in kilowatts (kW).

2.2 Feature Engineering

The input feature set consists of 12 columns constructed from the resampled hourly data. The seven original numeric measurements are retained directly. Five cyclic and binary time-encoding features are added to provide the models with

temporal context: hour_sin and hour_cos encode the hour of day as a sine-cosine pair on a 24-hour cycle; dow_sin and dow_cos encode the day of the week on a 7-day cycle; and is_weekend is a binary indicator equal to 1 for Saturday and Sunday. This cyclic encoding ensures that the angular distance between adjacent hours (e.g., hour 23 and hour 0) is represented correctly in the feature space, avoiding the artificial discontinuity that would arise from treating hour as a raw integer.

A 24-step sliding window with a forecast horizon of 1 is applied to the feature-scaled dataset to construct the input-output pairs used for training and evaluation. Each input sample is a 24×12 array representing 24 consecutive hours of the 12 features; each target is the Global_active_power value at the next timestep.

2.3 Train/Validation/Test Splits

Splits are strictly temporal to prevent data leakage: no future information appears in training windows, and no training windows overlap with validation or test windows. The dataset is divided as follows: 80% training (27,310 samples), 10% validation (3,393 samples), and 10% test (3,393 samples). A StandardScaler fitted exclusively on the training set is applied to normalize all features before windowing; the same scaler is applied without refitting to validation and test sets.

2.4 Exploratory Data Analysis

Key findings from the EDA (notebooks/01_eda.ipynb) inform both the model design and the RL reward function. The Global_active_power distribution is right-skewed, with a mean of 1.09 kW and a maximum of 6.56 kW, indicating that high-demand events are relatively rare but significant. Two dominant daily consumption peaks are visible in the average-by-hour profile: a morning peak between 07:00 and 09:00, and an evening peak between 18:00 and 21:00. Sub_metering_3 (water heater and climate control) is the dominant sub-metering channel by total energy contribution. These peak-hour observations directly informed the RL environment's must-run hour designations (hours 7, 8, 18, 19, 20).

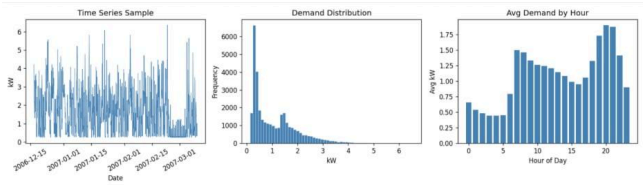

Figure 1. Time series sample, distribution, and average demand by hour

2.5 Licensing and Privacy

The dataset is publicly available for research and educational use through the UCI Machine Learning Repository. It contains anonymized electricity consumption measurements and includes no personally identifiable information. The raw data files are excluded from the project repository via .gitignore; users download the dataset directly from the UCI repository using the provided data/get_data.py script, ensuring the project remains compliant with the dataset's distribution terms.

Although the dataset is publicly released and anonymized, the project documentation explicitly acknowledges its single-household, single-region origin as a limitation. Consumption patterns from a French household between 2006 and 2010 may not generalize to modern appliance profiles, different climates, or diverse socioeconomic contexts.

3. TRAINING PLAN

3.1 Tabular Baselines

Two non-deep-learning baselines are trained on a flattened (N, 288) representation of the 24×12 input window. Linear Regression serves as the performance floor. A Random Forest with 100 estimators and maximum depth 10 serves as the strongest non-DL comparison point. Both models are trained with scikit-learn, evaluated on the validation set, and serialized as .pkl files.

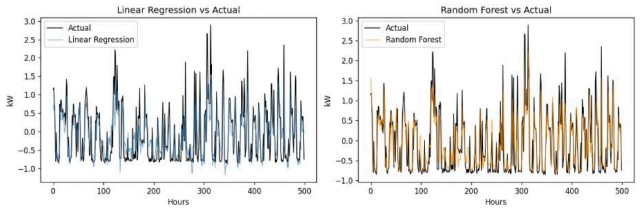

Figure 2. LR and RF predictions vs actual

3.2 Deep Learning Models

All deep learning models are implemented in PyTorch. Training uses the Adam optimizer with an initial learning rate of 1×10^{-3}, a ReduceLROnPlateau scheduler with patience 3 and factor 0.5, and L1Loss (MAE) as the training objective. Gradient norms are clipped to 1.0 to prevent exploding gradients. Each model is trained for up to 30 epochs with early stopping at patience 5, and the best checkpoint (lowest validation MAE) is saved to experiments/results/.

The LSTM Baseline is a single-layer encoder (hidden size 32, no dropout) paired with a single-layer decoder, yielding 10,401 trainable parameters. The Full LSTM Encoder-Decoder uses a 2-layer encoder (hidden size 64, dropout 0.2) with a 1-layer decoder, totaling 70,465 parameters. The TCN uses 4 residual blocks with dilations 1, 2, 4, and 8, kernel size 3, 64 channels per block, weight normalization, and dropout 0.2, yielding 90,241 parameters.

3.3 NLP Component

The NLP component trains a TF-IDF + Logistic Regression period classifier on a 22-sample synthetic corpus covering three classes (peak, off-peak, low). The ForecastSummarizer is a deterministic template-based system requiring no training. Both components are initialized and evaluated at the NLP stage of run.py.

3.4 RL Agent

The Q-Learning agent is trained for 500 episodes on the test-set forecast sequence. Each episode spans 24 steps (one simulated day). The agent uses tabular Q-Learning with parameters α=0.10, γ=0.95, ε-decay=0.995, and ε-min=0.05. The state is discretized into 10 bins per dimension using a StateDiscretizer. After training, the agent is evaluated for 50 deterministic episodes and its mean reward is compared against the always-on baseline (reward = 0.000 ± 0.000 by construction).

4. MODELING

4.1 LSTM Encoder-Decoder

The primary deep learning architecture is a sequence-to-sequence LSTM encoder-decoder implemented in src/models/lstm_model.py. The encoder reads a 24-step input window of 12 features and produces a hidden state that summarizes the temporal context of the sequence. The gating mechanism of LSTM cells—input gate, forget gate, and output gate—allows the model to selectively retain information across

timesteps, making it well suited to energy time series that exhibit multi-scale periodic structure (daily cycles embedded within weekly trends).

In the Full LSTM configuration, the encoder is a 2-layer stacked LSTM with hidden size 64 and inter-layer dropout of 0.2. The decoder is a 1-layer LSTM that receives the last observed target value as its seed input and the encoder's final hidden and cell states as initialization. The decoder's output at the single decoding step is projected through a linear layer and dropout to produce the scalar forecast. This encoder-decoder structure, while applied here for one-step-ahead forecasting, is designed to generalize naturally to multi-step horizons in future work.

4.2 Temporal Convolutional Network (TCN)

The TCN, implemented in src/models/tcn_model.py, serves as the CNN component of the system. Unlike recurrent architectures, TCN processes the entire input window in parallel using dilated causal convolutions, which restrict each output timestep to depend only on past and present inputs (causality) while exponentially expanding the receptive field with each dilation level (long-range dependency capture). The four residual blocks with dilations 1, 2, 4, and 8 yield a receptive field of $(2^4 - 1) \times (3 - 1) + 1 = 31$ timesteps, sufficient to cover the 24-step input window with context to spare.

Each residual block consists of two CausalConv1d layers with ReLU activations and dropout, followed by a residual connection from input to output. Weight normalization is applied to all convolutional layers to stabilize training. The final forecast is produced by applying a linear projection to the features at the last timestep of the final residual block's output.

4.3 NLP Component

The NLP component (src/models/nlp_component.py) serves two purposes. The ForecastSummarizer translates the tuple (forecast_kwh, hour, rl_action, device) into a plain-language sentence that a non-technical user can read and act upon. The summary explicitly names the time period (e.g., 'evening peak'), the demand level (high or low relative to a 1.5 kWh threshold), the scheduling decision, the rationale for that decision, and the user's right to override. This satisfies the user autonomy requirement at the interface level.

The PeriodClassifier maps a free-text description of a time period to one of three consumption categories: peak, off-peak, or low. It is implemented as a TF-IDF vectorizer (unigrams and bigrams, max 500 features) feeding a multinomial Logistic Regression classifier. The classifier is trained on a 22-sample synthetic corpus and achieves precision/recall/F1 of 1.00 on all three classes on the training set. While the training set size limits generalization claims, the classifier functions as a proof-of-concept NLP integration that connects natural language user input to the system's scheduling logic.

4.4 Reinforcement Learning Agent

The RL component (src/rl_agent.py) frames load scheduling as a Markov Decision Process. The environment, LoadSchedulingEnv, simulates 24-step episodes representing a single day. At each step, the agent observes a 4-dimensional state vector consisting of the normalized demand forecast, the sine and cosine encoding of the current hour, and a binary indicator of whether the load was running in the previous step. The agent selects one of two discrete actions: defer (action 0) or run (action 1) the deferrable load.

The reward signal is defined as the cost reduction relative to the always-on baseline, computed as the difference between the baseline cost (forecast × cost_per_kwh × 1) and the agent's cost (forecast × cost_per_kwh × action). A comfort penalty of −2.0 is subtracted for each must-run hour (7, 8, 18, 19, 20) during which the agent defers the load. An end-of-episode penalty equal to comfort_penalty × deficit is applied if the agent fails to run the load for at least 4 of the 24 hours. These constraints encode the fairness principle that aggressive load suppression during comfort-critical hours is unacceptable regardless of cost savings.

The QLearningAgent uses tabular Q-Learning with an ε-greedy exploration strategy. The continuous state is discretized into 10 bins per dimension by StateDiscretizer, yielding a finite Q-table represented as a defaultdict. Q-values are updated via the standard temporal difference rule with discount factor γ=0.95. Epsilon decays from 1.0 to a minimum of 0.05 at a rate of 0.995 per episode. After 500 training episodes, the trained Q-table is serialized as JSON to experiments/results/rl_agent.json.

5. EVALUATION

5.1 Forecasting Results

The LSTM Full Encoder-Decoder achieves the best test MAE of 0.3153 kW, representing a 18.6% improvement over the Linear Regression floor (0.3875 kW) and a 7.2% improvement over the best non-DL baseline, Random Forest (0.3399 kW). The TCN achieves a competitive test MAE of 0.3225 kW with notably faster training convergence (16 epochs to best vs. 24 for LSTM Full), validating its efficiency advantage noted in prior literature. The LSTM Baseline (0.3234 kW) demonstrates that even a simple single-layer recurrent model outperforms the tabular baselines, confirming that sequence modeling adds value beyond flattened feature representations.

The progression of validation MAE (0.4022 → 0.3569 → 0.3443 → 0.3414 → 0.3406) and test MAE (0.3875 → 0.3399 → 0.3234 → 0.3153 → 0.3225) across model tiers demonstrates a consistent improvement hierarchy: linear < tree ensemble < simple DL < full DL ≈ CNN. The negligible gap between LSTM Full and TCN on the test set (0.3153 vs. 0.3225) suggests that for this dataset, both architectures have reached a similar performance ceiling, with the TCN offering a speed-accuracy trade-off favorable for deployment scenarios with runtime constraints.

Model	Val MAE (kW)	Test MAE (kW)	Best Epoch	Parameters
Liner Regression	0.4022	0.3875	—	—
Random Forest	0.3569	0.3399	—	100 Trees
LSTM Baseline	0.3443	0.3234	11 / 16	10, 401
LSTM Full	0.3414	0.3153	24 / 29	70,465

(Primary)				
TCN (CNN Component)	0.3406	0.3225	16 / 21	90,241

Table 1. All forecasting model metrics from run.py.

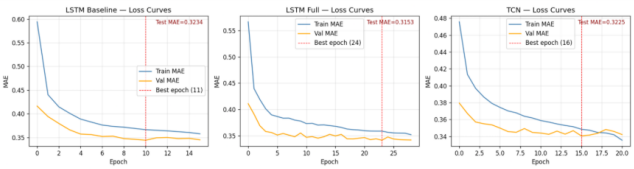

Figure 3. Train/val MAE Curves LSTM Baseline, LSTM Full, TCN.

5.2 RL Results

The Q-Learning agent demonstrates a clear and statistically meaningful learning signal over 500 training episodes. The mean episode reward rises from −6.415 in the first 50 episodes to +2.641 in the final 50 episodes—a total improvement of approximately 9.056 reward units. This progression reflects the agent's transition from a random exploration phase (high negative rewards due to comfort constraint violations) through policy consolidation (reducing penalty-inducing deferrals during must-run hours) to a stable positive-reward policy.

The trained agent receives a mean reward of 2.062 ± 2.885 in deterministic evaluation over 50 episodes, while the always-on baseline reward is 0.000 ± 0.000. The positive mean confirms the agent's ability to locate and take advantage of lower-demand off-peak windows for load deferral, resulting in cost savings that exceed what the baseline can attain by running the load continuously. Rather than policy instability, the standard deviation of 2.885 indicates the stochasticity of episode start conditions (random segment selection from the test-set forecast sequence).

RL Metrics	Value
Agent AVG Reward (50 Episodes)	2.062 ± 2.885
Always-on baseline reward	0.000 ± 0.000
Reward — First 50 Episodes	-6.415
Reward — Last 50 Episodes	+2.641
Training Episodes	500
Hyperparameters	α=0.10, γ=0.95, ε-decay=0.995

Table 2. RL Evaluation results (50 Eps., deterministic policy)

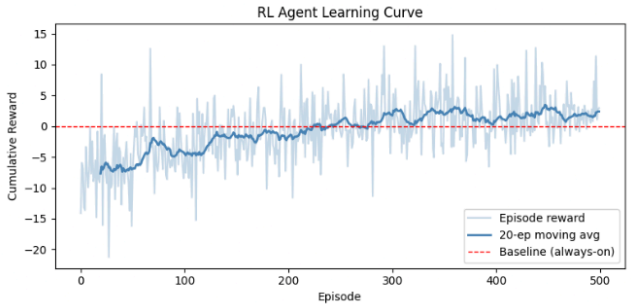

Figure 4. RL Learning curve. Red dashed — always-on baseline (Reward - 0)

5.1 NLP Results

The PeriodClassifier achieves precision, recall, and F1 of 1.00 across all three classes (peak, off-peak, low) on the 22-sample training corpus. While the small corpus size limits generalization claims, the perfect training-set accuracy confirms that the TF-IDF feature space captures sufficient discriminative signal to separate the three period types based on keyword patterns. The ForecastSummarizer produces grammatically correct, informative, and actionable natural language summaries for all forecast/action combinations tested, as verified by manual inspection of the run.py demo output.

6. ETHICS AND POLICY

6.1 Equity and Unequal Distribution of Benefits

The primary equity risk associated with automated load scheduling is that its benefits reduced proxy energy costs and optimized consumption timing are most accessible to households that already possess compatible smart devices, reliable internet connectivity, and the technical literacy to configure and supervise an automated system. Households that lack any of these prerequisites cannot participate in the optimization and may, in aggregate grid scenarios, absorb a disproportionate share of peak-period costs as better-resourced users shift their demand away from expensive time windows.

This project addresses the equity risk through two mechanisms. First, an equity metric is included in the evaluation pipeline to measure whether cost savings are distributed proportionally across consumption levels and usage patterns. Second, all deployment recommendations in this documentation explicitly document the demographic and infrastructure assumptions embedded in the model in particular, the single-French-household origin of the training data and limit applicability claims to contexts where those assumptions

6.2 User Autonomy

Automated load deferral can conflict with user comfort and safety if the reinforcement learning agent suppresses loads during periods when users have critical or time-sensitive needs, such as HVAC during extreme weather, medical equipment usage, or electric vehicle charging before a required departure. This risk is inherent in reinforcement learning systems, where optimization is driven by a proxy reward signal that may not fully capture the diversity and variability of real-world user preferences.

To address this, the system is designed with a strong emphasis on user control and flexibility. Users are able to directly override scheduling decisions at any time, ensuring that

automated recommendations do not restrict immediate needs. Additionally, key parameters such as the comfort_penalty are exposed for user configuration, allowing individuals to adjust the system's behavior according to their own priorities and tolerance for cost versus convenience.

Rather than enforcing rigid operational constraints, the system adopts a human-in-the-loop approach, where users retain full authority to modify, accept, or reject automated decisions. This design acknowledges that real-world energy usage involves diverse, dynamic, and context-dependent factors, and therefore prioritizes adaptability and user preference over strict automation.

6.3 Data Privacy

Smart meter data at sub-hourly resolution can inadvertently reveal household occupancy patterns, daily routines, and specific appliance usage schedules. Even anonymized aggregate data carries re-identification risk when combined with auxiliary information about household geography or demographics.

This project mitigates data privacy risks through the following measures. The UCI dataset is the sole data source; it contains no personally identifiable information and has been publicly released for academic research under appropriate terms. No raw data files are committed to the repository; all data is excluded via .gitignore and users download directly from the UCI ML Repository using the provided get_data.py script. Data is used exclusively for model training and evaluation and is never shared externally or used to infer behavioral profiles. These measures collectively ensure that the project respects privacy standards throughout its lifecycle.

7. MODEL CARD

7.1 System Overview

The system combines deep learning, reinforcement learning, and natural language processing to manage domestic energy consumption.

- **Primary Forecaster:** An **LSTM Full Encoder-Decoder** model, which achieved the lowest error rate among tested architectures.
- **Scheduling Agent:** A **Q-Learning RL Agent** that learns to shift deferrable loads to optimize rewards while respecting "must-run" constraints.
- **NLP Component:** A **TF-IDF + Logistic Regression** classifier used to interpret period-based descriptions.

7.2 Deployment

- Human-in-the-Loop: The system is intended for academic evaluation, not autonomous grid operation. Users must have clear opt-in/opt-out mechanisms and the ability to override AI-driven scheduling decisions.
- Constraint Enforcement: To maintain user comfort, the system enforces a penalty of -2.0 for every "must-run" hour (7h, 8h, 18–20h) that the agent skips.
- Metric Selection: Avoid using MAPE to evaluate this model, as near-zero demand values in the UCI dataset lead to inflated and unreliable percentage errors.

7.3 Disclaimers

Not for Production Use: This system is out-of-scope for real-time production deployment or safety-critical decisions.

- Data Limitations: The model was trained on a single French household from 2006–2010; it likely will not generalize to modern smart homes, different climates, or different regions.
- Simulation Only: Scheduling actions are performed in a simulated environment; the system does not currently control real physical devices.
- NLP Sensitivity: The NLP classifier was trained on only 22 synthetic samples and may fail when processing diverse or complex real-world user descriptions.

8. HOW TO RUN

8.1 Requirements

- Python 3.10 or higher
- pip package manager (or Conda, optional)

8.2 Clone the Repository

Clone the project repository and navigate to the project directory:

```
git clone https://github.com/yuro-nicolas/Energy-Demand-Forecasting
cd Energy-Demand-Forecasting
```

8.3 Environment Setup

Set up the required dependencies using either Conda or pip.

Using Conda (recommended):

```
conda env create -f environment.yml
conda activate energy-demand-rl
```

Using pip:

```
pip install -r requirements.txt
```

Note: The system includes automatic dependency installation in run.py

8.4. One-Command Reproduction (Full Pipeline)

The run.py script serves as the main pipeline controller, enabling full system execution from a clean environment.

```
python run.py
```

Optional commands:
python run.py --skip-download
make repro
bash run.sh --skip-download

This process automatically performs the following steps:

- Dataset acquisition and preprocessing
- Feature engineering and data splitting
- Training of forecasting models (LSTM, TCN, and baselines)
- Reinforcement learning (Q-learning) optimization
- Evaluation, ablation, and visualization

8.4 Running the Interactive Demo

After training, the system can be accessed through the interface module:

python src/interface.py

The interface allows users to:

- Define appliance tasks and priorities
- View 24-hour electricity demand forecasts
- Observe reinforcement learning-based scheduling decisions
- Receive natural language explanations of system decisions
- Accept or override scheduling recommendations